

# 채용 시스템

## 시나리오

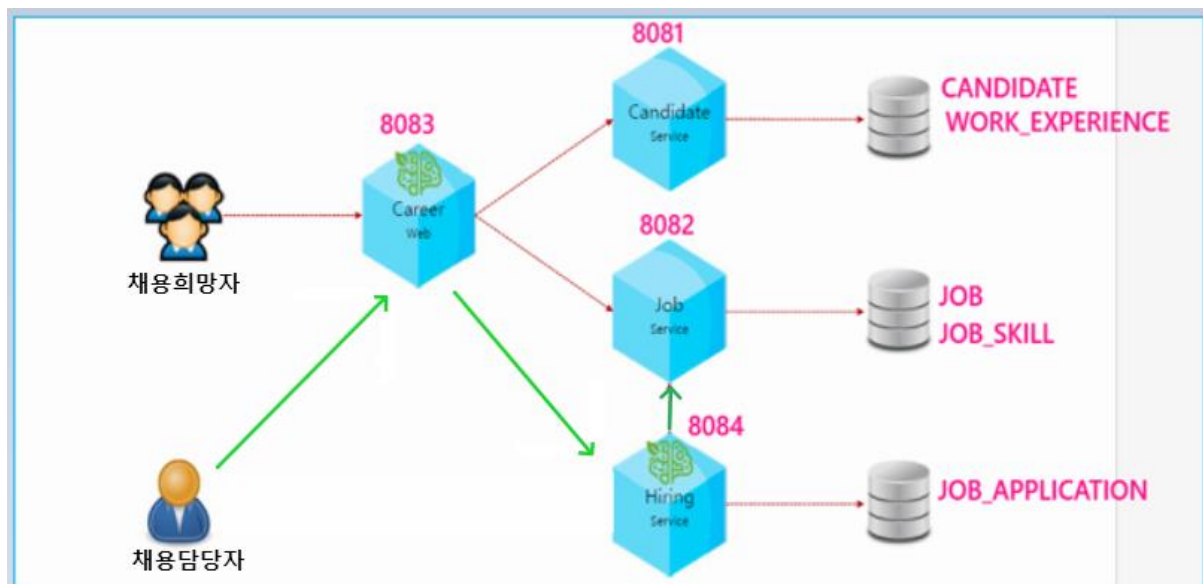
### - 채용희망자 관점

- 1) 나의 프로파일에 적합한 JOB을 AI가 판단하여 추천
- 2) AI가 추천한 JOB들에 대한 비교 제공
- 3) 내가 지원하려는 회사에 맞게 AI가 이력서 작성

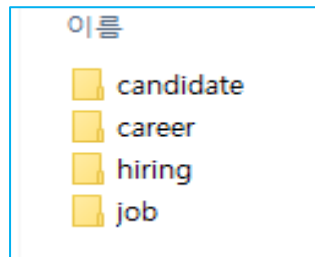
### - 채용 담당자 관점

- 1) JOB에 지원한 "채용 희망자"가 우리 회사에 적합한지 AI가 판단

## AI기반 채용 시스템



## Spring Project 4개



## # candidate Project – REST API

Port : 8081

DB : h2 / data.sql제공

**CANDIDATE**

Id, name, email, phone, location, experience\_in\_years, skills

select \* from CANDIDATE;

| ID | NAME | EMAIL             | PHONE           | LOCATION          | EXPERIENCE_IN_YEARS | SKILLS                              |
|----|------|-------------------|-----------------|-------------------|---------------------|-------------------------------------|
| 1  | 정희정  | 8253jang@daum.net | +1-555-987-6543 | San Francisco, CA | 6                   | [Java, Spring Boot, Hibernate, AWS] |
| 2  | 이찬범  | chan@naver.com    | +1-555-234-5678 | Austin, TX        | 5                   | [Go, gRPC, Kubernetes, PostgreSQL]  |

(2 rows)

**WORK\_EXPERIENCE**

id , candidate\_id, company\_name, job\_title, start\_date, end\_date, description, technologies

select \* from WORK\_EXPERIENCE;

| ID | CANDIDATE_ID | COMPANY_NAME           | JOB_TITLE               | START_DATE | END_DATE   | DESCRIPTION                                                                                                                                                 | TECHNOLOGIES                                |
|----|--------------|------------------------|-------------------------|------------|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| 1  | 1            | TechSphere Inc.        | Senior Java Developer   | 2019-06-01 | 2023-07-31 | Led the design and development of backend microservices for a high-traffic e-commerce platform using Java and Spring Boot. Improved API performance by 30%. | [Java, Spring Boot, Hibernate, AWS, Docker] |
| 2  | 1            | CloudEdge Solutions    | Backend Developer       | 2016-05-01 | 2019-05-31 | Developed scalable REST APIs and implemented CI/CD pipelines.                                                                                               | [Java, Microservices, MySQL, Jenkins]       |
| 3  | 2            | StreamLogic Inc.       | GoLang Backend Engineer | 2019-08-01 | 2023-02-28 | Developed high-performance microservices using GoLang and gRPC.                                                                                             | [Go, gRPC, Kubernetes, Prometheus, Docker]  |
| 4  | 2            | NexiGen Cloud Services | Software Engineer       | 2017-06-01 | 2019-07-31 | Built cloud-native backend solutions with Go.                                                                                                               | [Go, REST API, PostgreSQL, Terraform]       |

(4 rows)

**build.gradle**

```
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-web'
    runtimeOnly 'com.h2database:h2'
}
```

**application.properties**

```

1      spring.application.name=candidate
2      server.port=8081
3
4       spring.datasource.url=jdbc:h2:mem:testdb
5      spring.datasource.driver-class-name=org.h2.Driver
6      spring.datasource.username=sa
7      spring.datasource.password=
8
9      spring.h2.console.enabled=true
10     spring.h2.console.path=/h2-console
11     
12     #spring.jpa.hibernate.ddl-auto=create
13     spring.jpa.defer-datasource-initialization=true
14
15     #spring.jpa.show-sql=true
16     spring.jpa.properties.hibernate.format_sql=true
17     logging.level.org.hibernate.SQL=debug
18     logging.level.org.hibernate.orm.jdbc.bind=trace

```

### CandidateRepository.java

```

@Repository 3 usages
public interface CandidateRepository extends JpaRepository<Candidate, Integer> {

} 

```

### CandidateService.java

```

8       @Service 3 usages
9      public class CandidateService {
10         private final CandidateRepository candidateRepository; 2 usages
11
12         > public CandidateService(CandidateRepository candidateRepository) { this.candidateRepository = candidateRepository; }
13
14
15         public CandidateDetails getCandidateDetails(Integer id) { 1 usage
16             return this.candidateRepository.findById(id).orElseThrow();
17             .map(EntityDtoMapper::toCandidateDetails) Optional<CandidateDetails>
18             .orElseThrow();
19         }
20     }
21
22

```

### CandidateController.java

```

9  @CrossOrigin
10 @RestController
11 @RequestMapping("/api/candidates")
12 public class CandidateController {
13
14     private static final Logger log = LoggerFactory.getLogger(CandidateController.class); 1 usage
15     private final CandidateService candidateService; 2 usages
16
17     public CandidateController(CandidateService candidateService) { this.candidateService = candidateService; }
18
19     @GetMapping("/{candidateId}")
20     public CandidateDetails getCandidateById(@PathVariable Integer candidateId) {
21         log.info("Fetching candidate by id: {}", candidateId);
22         return this.candidateService.getCandidateDetails(candidateId);
23     }
24 }

```

### rest-client.http 파일

IntelliJ IDEA Ultimate 버전이라면 **.http** 또는 **.rest** 확장자의 파일을 만들면 자동으로 HTTP Client 기능이 활성화

### 환경변수도 사용할 수 있음

```

<> http

@host = localhost
@port = 8083

### Find Jobs
GET http://{{host}}:{{port}}/api/career-advisor/find-jobs?candidateId=1

```

```

1  ### candidate-details
2  GET http://localhost:8081/api/candidates/1
3  ###
4  GET http://localhost:8081/api/candidates/2

```

### test결과

HTTP Request
 

All in rest-client
 

rest-client | #1
 

GET rest-client | #2

Docker

HTTP/1.1 200

> (Headers) ...Content-Type: application/json...

```

{
  "id": 1,
  "name": "장희정",
  "email": "8253jang@daum.net",
  "phone": "+1-555-987-6543",
  "location": "San Francisco, CA",
  "experienceInYears": 6,
  "skills": [
    "Java",
    "Spring Boot",
    "Hibernate",
    "AWS"
  ],
  "workExperiences": [
    {
      "companyName": "TechSphere Inc.",
      "jobTitle": "Senior Java Developer",
      "startDate": "2019-06-01",
      "endDate": "2023-07-31",
      "description": "Led the design and development of backend microservices for a high",
      "technologies": [
        "Java",
        "Spring Boot",
        "Hibernate",
        "AWS"
      ]
    }
  ]
}

```

## # JOB Project – REST API

Port : 8082

DB : h2 / data.sql제공

### JOB

id, title, description, location, is\_remote, salary\_min, salary\_max, POSTED\_DATE, employer

| SELECT * FROM JOB; |                                   |                                                                                   |                   |           |            |            |                           |                       |
|--------------------|-----------------------------------|-----------------------------------------------------------------------------------|-------------------|-----------|------------|------------|---------------------------|-----------------------|
| ID                 | TITLE                             | DESCRIPTION                                                                       | LOCATION          | IS_REMOTE | SALARY_MIN | SALARY_MAX | POSTED_DATE               | EMPLOYER              |
| 1                  | Senior Software Engineer - Java   | Lead the design and development of backend services using Java and Spring Boot.   | New York, NY      | TRUE      | 105000     | 140000     | 2026-06-08 16:41:21.16561 | FinEdge Technologies  |
| 2                  | Backend Developer - Golang        | Develop high-performance microservices in Go for large-scale distributed systems. | Remote            | FALSE     | 95000      | 125000     | 2026-06-08 16:41:21.16561 | CloudWorks Labs       |
| 3                  | Platform Engineer - Java          | Own backend platform services powering millions of API requests daily.            | Austin, TX        | FALSE     | 98000      | 130000     | 2026-06-08 16:41:21.16561 | DataMesh Corp.        |
| 4                  | Software Engineer II (Golang)     | Build secure and scalable APIs for real-time applications using Go.               | Remote            | TRUE      | 90000      | 115000     | 2026-06-08 16:41:21.16561 | GoScale Systems       |
| 5                  | Senior Java Backend Engineer      | Architect and deliver cloud-native backend solutions using Java and Spring Cloud. | San Francisco, CA | FALSE     | 120000     | 150000     | 2026-06-08 16:41:21.16561 | EdgeWave Solutions    |
| 6                  | Principal Software Engineer (Go)  | Drive backend architecture decisions and mentor team in Golang best practices.    | Seattle, WA       | FALSE     | 130000     | 160000     | 2026-06-08 16:41:21.16561 | StreamLogic Inc.      |
| 7                  | Backend Services Developer - Java | Implement business-critical backend services using Java and Hibernate.            | Boston, MA        | FALSE     | 85000      | 105000     | 2026-06-08 16:41:21.16561 | BrightTech Consulting |

**컬럼 정의**

| 컬럼명         | 데이터 의미       | 예시                              |
|-------------|--------------|---------------------------------|
| ID          | 채용공고 식별자(PK) | 1                               |
| TITLE       | 채용공고 제목      | Senior Software Engineer - Java |
| DESCRIPTION | 업무 내용 및 요구사항 | Java/Spring Boot 백엔드 개발         |
| LOCATION    | 근무지          | New York, NY                    |
| IS_REMOTE   | 원격 근무 여부     | TRUE / FALSE                    |
| SALARY_MIN  | 최소 연봉        | 105000                          |
| SALARY_MAX  | 최대 연봉        | 140000                          |
| POSTED_DATE | 공고 게시일       | 2026-06-17                      |
| EMPLOYER    | 채용 기업명       | FinEdge Technologies            |

**JOB\_SKILL**

id , job\_id, skill

SELECT \* FROM JOB\_SKILL;

| ID | JOB_ID | SKILL        |
|----|--------|--------------|
| 1  | 1      | Java         |
| 2  | 1      | Spring Boot  |
| 3  | 1      | Docker       |
| 4  | 1      | AWS          |
| 5  | 2      | Go           |
| 6  | 2      | gRPC         |
| 7  | 2      | PostgreSQL   |
| 8  | 2      | Kubernetes   |
| 9  | 3      | Java         |
| 10 | 3      | Spring Cloud |

**build.gradle**

```
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-web'
    runtimeOnly 'com.h2database:h2'
}
```

**application.properties**

```

1  spring.application.name=job
2  server.port=8082
3  spring.jpa.defer-datasource-initialization=true
4
5  spring.datasource.url=jdbc:h2:mem:testdb
6  spring.datasource.driver-class-name=org.h2.Driver
7  spring.datasource.username=sa
8  spring.datasource.password=
9
10 spring.h2.console.enabled=true
11 spring.h2.console.path=/h2-console
12
13 spring.jpa.properties.hibernate.format_sql=true
14 logging.level.org.hibernate.SQL=debug
15 logging.level.org.hibernate.orm.jdbc.bind=trace

```

### JobRepository.java

```

@Repository 3 usages
public interface JobRepository extends JpaRepository<Job, Integer> {

    @Query(""" 1 usage
        SELECT DISTINCT j
        FROM Job j
        JOIN j.skills s
        WHERE LOWER(s.skill) IN :skills
        """)
    List<Job> findByAnySkill(List<String> skills);

}

```

### JobService.java

```
11 @Service 3 usages
12 public class JobService {
13     private final JobRepository jobRepository; 4 usages
14
15     > public JobService(JobRepository jobRepository) { this.jobRepository = jobRepository; }
16
17
18
19     public JobDetails getJobById(Integer id) { 1 usage
20         return this.jobRepository.findById(id) Optional<Job>
21             .map(EntityDtoMapper::toJobDetails) Optional<JobDetails>
22             .orElseThrow();
23     }
24
25     public List<JobDetails> getJobsByIds(List<Integer> ids) { 1 usage
26         return this.jobRepository.findAllById(ids) List<Job>
27             .stream() Stream<Job>
28             .map(EntityDtoMapper::toJobDetails) Stream<JobDetails>
29             .toList();
30     }
31
32     @ public List<JobSummary> searchBySkills(List<String> skills) 1 usage
33         var lowercasedSkills = skills.stream()
34             .map(String::toLowerCase)
35             .toList();
36         return this.jobRepository.findByAnySkill(lowercasedSkills) List<Job>
37             .stream() Stream<Job>
38             .map(EntityDtoMapper::toJobSummary) Stream<JobSummary>
39             .toList();
40     }
41
42 }
```

## JobController.java



```

12  @CrossOrigin
13  @RestController
14  @RequestMapping("/api/jobs")
15  public class JobController {
16
17      private static final Logger log = LoggerFactory.getLogger(JobController.class);
18      private final JobService jobService; 4 usages
19
20      public JobController(JobService jobService) {
21          this.jobService = jobService;
22      }
23
24      @GetMapping("/{id}")
25      public JobDetails getJobById(@PathVariable Integer id){
26          log.info("Fetching job by id: {}", id);
27          return this.jobService.getJobById(id);
28      }
29
30      @GetMapping(params = "ids")
31      public List<JobDetails> getJobsByIds(@RequestParam List<Integer> ids){
32          log.info("Fetching jobs by ids: {}", ids);
33          return this.jobService.getJobsByIds(ids);
34      }
35
36      @GetMapping(params = "skills")
37      public List<JobSummary> searchBySkills(@RequestParam List<String> skills){
38          log.info("Searching jobs by skills: {}", skills);
39          return this.jobService.searchBySkills(skills);
40      }
41
42  }

```

### rest-client.http 파일

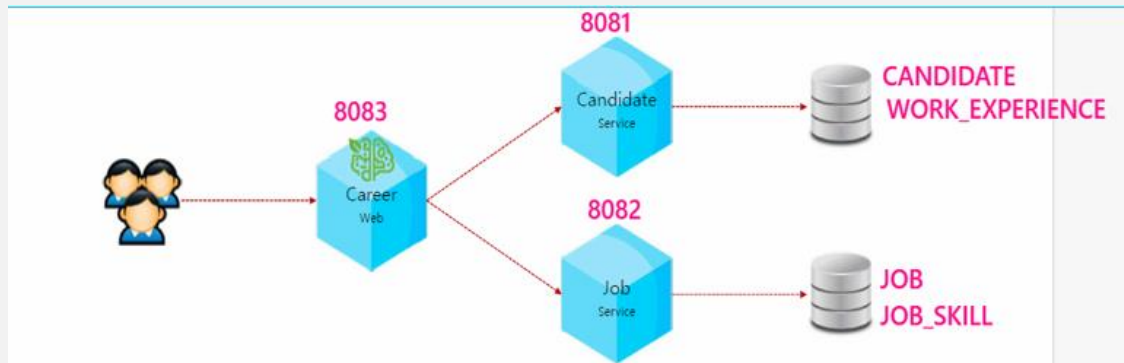
IntelliJ IDEA Ultimate 버전이라면 **.http** 또는 **.rest** 확장자의 파일을 만들면 자동으로 HTTP Client 기능이 활성화



## # Career Project

Port : 8083

DB : 없음



build.gradle

```

dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation 'org.springframework.ai:spring-ai-starter-model-google-genai'
    implementation 'org.springframework.ai:spring-ai-starter-model-ollama'
    implementation 'org.springframework.ai:spring-ai-starter-model-openai'
    implementation 'org.springframework.boot:spring-boot-starter-actuator'
    implementation 'tools.jackson.core:jackson-databind:3.1.3'
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}
  
```

→ 애플리케이션의 상태를 모니터링하고 관리하기 위한 기능

Spring Boot의 Actuator(액추에이터) 참고

<https://app.notion.com/p/Spring-Boot-Actuator-37aa7a6c42ce80adb8f5d0b203e33>

application.properties

The screenshot shows an IDE with three files open: `application.properties`, `CareerAdvisorController.java`, and `application-openai.properties`. The `application.properties` file contains the following properties:

```
spring.application.name=career
server.port=8083
candidate-service.url=http://localhost:8081
job-service.url=http://localhost:8082
```

The `application-openai.properties` file contains the following properties:

```
spring.ai.model.chat=openai
spring.ai.openai.chat.options.model=gpt-5.4-nano
spring.ai.openai.api-key=${OPENAI_API_KEY}
```

The `CareerAdvisorController.java` file contains the following code:

```
@Bean
public JobClient jobClient(RestClient.Builder builder, @Value("${job-service.url}") String baseUrl){
    var restClient = builder.baseUrl(baseUrl).build();
    return new JobClient(restClient);
}

@Bean
public CandidateClient candidateClient(RestClient.Builder builder, @Value("${candidate-service.url}") String baseUrl){
    var restClient = builder.baseUrl(baseUrl).build();
    return new CandidateClient(restClient);
}
```

```
@Bean
public JobClient jobClient(RestClient.Builder builder, @Value("${job-service.url}") String baseUrl){
    var restClient = builder.baseUrl(baseUrl).build();
    return new JobClient(restClient);
}

@Bean
public CandidateClient candidateClient(RestClient.Builder builder, @Value("${candidate-service.url}") String baseUrl){
    var restClient = builder.baseUrl(baseUrl).build();
    return new CandidateClient(restClient);
}
```

## view구성 - html기반

- static
  - <> career-advisor-candidate.html
  - <> career-advisor-recruiter.html
  - <> index.html

## 주요기능

- 1 **Find Jobs**
- 2 GET <http://localhost:8083/api/career-advisor/find-jobs?candidateId=1>
- 3
- 4 **Compare Jobs**
- 5 GET <http://localhost:8083/api/career-advisor/compare-jobs?candidateId=1&jobIds=1,2>
- 6
- 7 **Generate Resume**
- 8 GET <http://localhost:8083/api/career-advisor/generate-resume?candidateId=1&jobId=1>

<http://localhost:8083/>

➔ 후보자 id에 해당하는 정보 조회

TalentFlow AI

Candidate View Recruiter View

Career Advisor

My Profile

DEMO CONTEXT: 1 Switch Candidate

career-advisor-candidate.html 문서 - loadProfile() 함수에서 fetch 함수로 candidate rest api 요청 해서 결과 화면에 렌더링

**장희정**

San Francisco, CA  
+1-555-987-6543  
8253jang@daum.net

6 Years Professional Experience

**Core Skills**

Java Spring Boot Hibernate AWS

**Experience History**

**Senior Java Developer @ TechSphere Inc.**  
2019-06-01 - 2023-07-31  
Led the design and development of backend microservices for a high-traffic e-commerce platform using Java and Spring Boot. Improved API performance by 30%.

Java Spring Boot Hibernate AWS Docker

**Backend Developer @ CloudEdge Solutions**  
2016-05-01 - 2019-05-31  
Developed scalable REST APIs and implemented CI/CD pipelines.

Java Microservices MySQL Jenkins

localhost:8081/api/candidates/1

pretty print 적용

```
{
  "id": 1,
  "name": "장희정",
  "email": "8253jang@daum.net",
  "phone": "+1-555-987-6543",
  "location": "San Francisco, CA",
  "experienceInYears": 6,
  "skills": [
    "Java",
    "Spring Boot",
    "Hibernate",
    "AWS"
  ],
  "workExperiences": [
    {
      "companyName": "TechSphere Inc.",
      "jobTitle": "Senior Java Developer",
      "startDate": "2019-06-01",
      "endDate": "2023-07-31",
      "description": "Led the design and development of backend microservices for a high-traffic e-commerce platform using Java and Spring Boot. Improved API performance by 30%.",
      "technologies": [
        "Java",
        "Spring Boot",
        "Hibernate",
        "AWS",
        "Docker"
      ]
    },
    {
      "companyName": "CloudEdge Solutions",
      "jobTitle": "Backend Developer",
      "startDate": "2016-05-01",
      "endDate": "2019-05-31",
      "description": "Developed scalable REST APIs and implemented CI/CD pipelines.",
      "technologies": [
        "Java",
        "Microservices",
        "MySQL",
        "Jenkins"
      ]
    }
  ]
}
```

```

async function loadProfile() {
    const cid = document.getElementById('candidate-id').value;

    try {
        const res = await fetch(`${CANDIDATE_SERVICE_URL}/api/candidates/${cid}`); //
        const data = await res.json();
        renderProfileCard(data);
    } catch (e) {
        console.error("Failed to load profile", e);
    }
}

```

```

function renderProfileCard(data) {
    const experienceHtml = data.workExperiences.map(exp => `

```

ateDetails.java    WorkExperience.java    **CandidateController.java**    application.properties

```

9      @CrossOrigin
10     @RestController
11     @RequestMapping("/api/candidates")
12     public class CandidateController {
13
14         private static final Logger log = LoggerFactory.getLogger(CandidateController.class);
15         private final CandidateService candidateService;
16
17         public CandidateController(CandidateService candidateService) {
18             this.candidateService = candidateService;
19         }
20
21         @GetMapping("/{candidateId}")
22         public CandidateDetails getCandidateById(@PathVariable Integer candidateId) {
23             log.info("Fetching candidate by id: {}", candidateId);
24             return this.candidateService.getCandidateDetails(candidateId);
25         }
26     }

```

```

@Service
public class CandidateService {
    private final CandidateRepository candidateRepository;

    public CandidateService(CandidateRepository candidateRepository) {
        this.candidateRepository = candidateRepository;
    }

    public CandidateDetails getCandidateDetails(Integer id) {
        return this.candidateRepository.findById(id)
            .map(EntityDtoMapper::toCandidateDetails)
            .orElseThrow();
    }
}

```

<http://localhost:8083/api/career-advisor/find-jobs?candidateId=1>

➔ 후보자의 skills정보와 각 일자리와의 매칭 점수를 기반으로 추천

## 매칭점수 기반 추천에 AI적용

The screenshot shows the 'Recommended Jobs' section of the TalentFlow AI Career Advisor. It displays three job recommendations with their match scores and AI insights:

- Senior Software Engineer - Java** (86% Match): FinEdge Technologies • New York, NY. AI Insight: 귀하는 Java/Spring Boot 기반 백엔드 마이크로서비스를 시니어급으로 설계·개발했고 성능 개선(30%)과 AWS 운영 경험이 있어 기술 적합도가 높습니다. 다만 근무지는 샌프란시스코로 제시되어 있어 뉴욕 현지/전환 가능성 확인이 필요하며, 공고가 요구하는 추가 스택(예: 프레임워크/데이터베이스/클라우드 범위)이 무엇인지에 따라 격차가 생길 수 있습니다.
- Backend Services Developer - Java** (83% Match): BrightTech Consulting • Boston, MA. AI Insight: 귀하 REST API와 CI/CD(Jenkins) 및 마이크로서비스를 구현한 경험이 있어 'Backend Services Developer - Java'에 직접적으로 부합합니다. 다만 AWS/Docker 중심의 경험은 강점이지만, 보스턴 근무 전환(또는 원격 가능 여부)과 공고의 세부 요구사항(특정 프레임워크/DB 테스트/관측 체계)이 확인되어야 100%에 가까워집니다.
- Java Microservices Developer** (74% Match): 제이드크로스 • 서울, 강남. AI Insight: 귀하 Java 마이크로서비스 백엔드 역량(Spring Boot/Hibernate/AWS)이 명확해 해당 직무의 핵심 기술과 잘 맞습니다. 다만 근무지가 샌프란시스코에서 서울(강남)로 크게 이동해야 하므로 근무 가능 시점/비자/이주 계획이 가장 큰 격차이며, 언어/로컬 협업 요구가 있다면 추가 확인이 필요합니다.

Annotations on the screenshot:

- 1) 클릭: Points to the 'Job Opportunities' button in the left sidebar.
- 2) 클릭: Points to the 'Find Jobs' button in the top right corner.

localhost:8083/api/career-advisor/find-jobs?candidateId=1

pretty print 적용

```
{
  "jobId": 1,
  "title": "Senior Software Engineer - Java",
  "location": "New York, NY",
  "employer": "FinEdge Technologies",
  "matchScore": 86,
  "matchReasoning": "당신은 Java/Spring Boot 기반 백엔드 마이크로서비스를 선임급으로 설계·개발했고, 성능 개선(30%) 및 AWS 활용 경험이 있어 직무 핵심입니다.",
},
{
  "jobId": 7,
  "title": "Backend Services Developer - Java",
  "location": "Boston, MA",
  "employer": "BrightTech Consulting",
  "matchScore": 80,
  "matchReasoning": "당신은 REST API, 마이크로서비스, CI/CD(Jenkins) 경험이 있어 'Backend Services Developer - Java'의 실행 역량과 직접 중복됩니다.",
},
{
  "jobId": 9,
  "title": "Java Microservices Developer",
  "location": "서울, 강남",
  "employer": "제이드크로스",
  "matchScore": 68,
  "matchReasoning": "당신은 Java 마이크로서비스(Spring Boot, Hibernate)로 백엔드 서비스를 구축한 경험이 있어 기술 스택 측면에서 강하게 부합합니다. 그"
}
```

CareerAdvisorService.java   application.properties   CareerAdvisorController.java   index.html

```
public class CareerAdvisorController {
    @GetMapping("/find-jobs")
    public List<JobEvaluationResult> findJobs(@RequestParam Integer candidateId) {
        log.info("Finding jobs for candidateId: {}", candidateId);
        return this.careerAdvisorService.findJobs(candidateId);
    }
}
```

```

15 public class CareerAdvisorService {
16
17     public List<JobEvaluationResult> findJobs(Integer candidateId) { 1 usage
18         // 1. 후보자의 상세 정보를 가져온다.
19         var candidate = this.candidateClient.getCandidateDetails(candidateId);
20
21         //2. 후보자의 스킬셋을 기반으로 관련된 일자리들을 검색한다.
22         var jobs = this.jobClient.searchBySkills(candidate.skills()); //ex) [Java, Spring Boot, Hibernate, AWS]
23
24         //3. 각 일자리와 후보자의 매칭 점수를 계산한다.
25         return this.advisorClient.evaluateJobs(candidate, jobs) List<JobEvaluationResult>
26             .stream() Stream<JobEvaluationResult>
27             .sorted(Comparator.comparingInt(JobEvaluationResult::matchScore).reversed())
28             .toList();
29     }
30 }

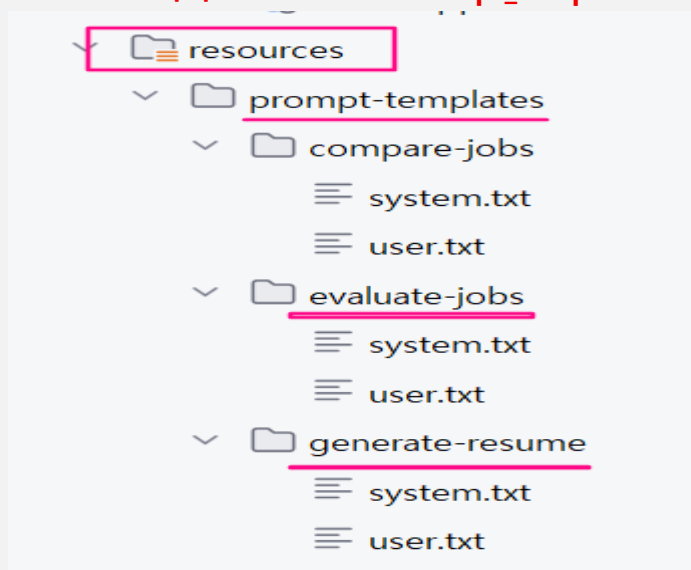
```

```

21 public class ApplicationConfiguration {
22
23     @Bean
24     public CareerAdvisorClient careerAdvisorClient(ChatClient.Builder builder, JsonMapper jsonMapper){
25         var chatClient = builder.build();
26         var prompts = new CareerAdvisorPrompts(
27             this.getPromptSet("compare-jobs"),
28             this.getPromptSet("evaluate-jobs"),
29             this.getPromptSet("generate-resume")
30         );
31         return new CareerAdvisorClient(chatClient, prompts, jsonMapper);
32     }
33
34     private PromptSet getPromptSet(String feature){ 3 usages
35         return new PromptSet(
36             this.getResourceContent(SYSTEM_TEMPLATE_PATH_FORMAT.formatted(feature)),
37             this.getResourceContent(USER_TEMPLATE_PATH_FORMAT.formatted(feature))
38         );
39     }
40
41     private String getResourceContent(String resourcePath){ 2 usages
42         try{
43             var resource = this.resourceLoader.getResource(resourcePath);
44             return resource.getContentAsString(Charset.defaultCharset());
45         } catch (IOException e) {
46             // ...
47         }
48     }
49 }

```

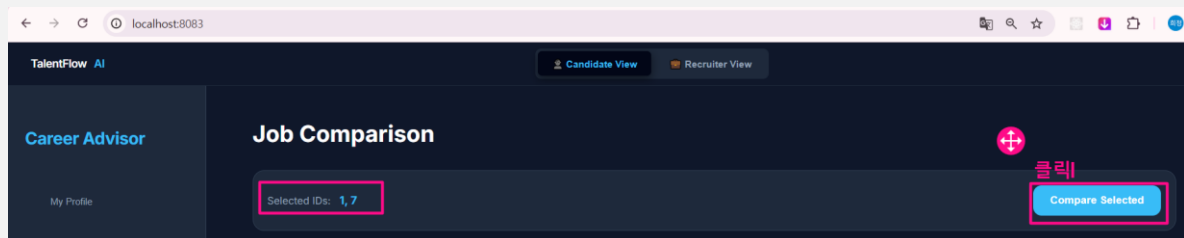
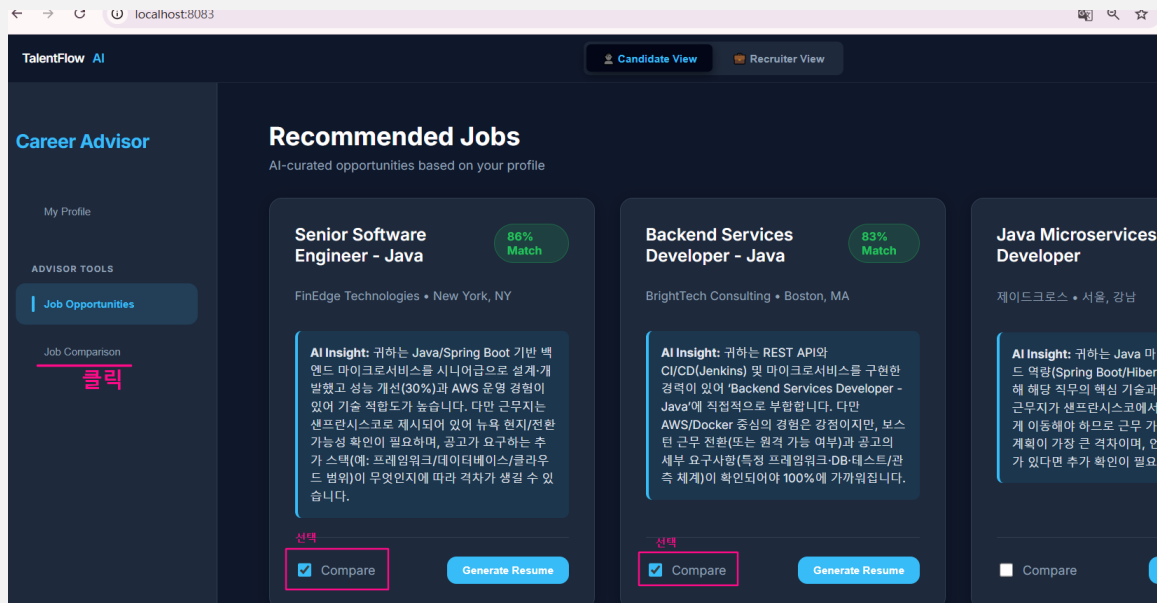
career프로젝트의 resources > Prompt\_templates.md 파일 참고





<http://localhost:8083/api/career-advisor/compare-jobs?candidateId=1&jobIds=1,2>

- ➔ 추천 받은 항목에서 체크박스를 선택 한 후 왼쪽의 Job Comparison 클릭하고  
오른쪽 상단의 Compare selected 클릭(AI가 추천한 JOB들에 대한 비교)



## Job Comparison

Selected IDs: 1, 7

Compare Selected

**Final Verdict:** 최적의 선택은 1번( Senior Software Engineer - Java )입니다. 기술적으로는 Java/Spring Boot/Docker/AWS 요구와의 스택 중복이 가장 크고, 원격 근무 옵션(isRemote=true)으로 근무지 배치 가능성도 가장 유리합니다. 7번도 Hibernate 강점은 맞지만, MySQL 요구와 온사이트 제약 때문에 전체 성장/적합도 관점에서 우선순위가 낮습니다.

| Feature               | 1: Senior Software Engineer - Java                                                                                                                                                             | 7: Backend Services Developer - Java                                                                                                                                                         |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Skills</b>         | <b>EXCELLENT</b><br>요구 스택(Java, Spring Boot, Docker, AWS)과 후보자 스택이 중복됩니다. 특히 Spring Boot/Java는 핵심이고, Docker와 AWS도 경력에 포함되어 기술 적합성이 가장 높습니다. (Hibernate는 요구에 직접 명시되진 않지만, 백엔드 전반 역량으로 간접 보완 가능) | <b>GOOD</b><br>후보자 스택 중 Java와 Hibernate는 요구와 정확히 일치합니다. 다만 요구 스택에 포함된 MySQL은 후보자 기술 목록/기술 태그에 나타나지 않고, Docker는 일부 경력에 있으나 (TechSphere 기술 스택에 Docker 포함) MySQL까지 완전히 커버되지는 않아 중독 커버리지가 제한됩니다. |
| <b>Location</b>       | 현재 거주지(샌프란시스코)와 직무 위치(뉴욕)가 다르지만, isRemote=true이므로 원격 근무로 지리적 제약을 상당 부분 상쇄할 수 있습니다.                                                                                                             | 직무 위치(보스턴)와 현재 거주지(샌프란시스코)가 달라 이사/온사이트 제약이 큼니다. isRemote=false이므로 원격 근무 가능성은 제공되지 않습니다.                                                                                                      |
| <b>Salary</b>         | 경력 6년 수준의 범위로 보이며, 제시 연봉은 \$105,000~\$140,000입니다(후보자 경력에 따른 정확한 개인 연봉 정보는 명시되지 않음).                                                                                                            | 제시 연봉은 \$85,000~\$105,000이며, 후보자 경력 6년을 감안하면 상단 보다는 중간~하단 범위일 가능성이 있습니다(개인 연봉 데이터는 명시되지 않음).                                                                                                 |
| <b>Recommendation</b> | 현재 경력(자바 기반 마이크로서비스, Spring Boot, Hibernate/백엔드, AWS)과 요구 스택이 가장 잘 맞고, 원격 옵션이 있어 근무지 리스크가 낮습니다.                                                                                                | Hibernate 중심의 백엔드 역할에 강점이 맞지만, 필수 스택에서 MySQL이 요구되고 Docker가 요구되는 반면 후보자의 MySQL 경험은 데이터에 없어서 적합성이 상대적으로 낮습니다.                                                                                  |

localhost:8083/api/career-advisor/compare-jobs?candidateId=1&amp;jobIds=1,2

pretty print 적용

```
{
  "overallVerdict": "가장 최적의 선택은 1번(Senior Software Engineer - Java)입니다. Java/Spring Boot 중심의 백엔드 마이크로서비스 경험이 요구기술과 4개 항목에서 직접 중복되고(AWS, Docker 포함)입니다.",
  "jobInsights": [
    {
      "jobId": 1,
      "jobTitle": "Senior Software Engineer - Java",
      "skillAlignment": "EXCELLENT",
      "skillInsight": "필수기술(Java, Spring Boot, Docker, AWS)과 당신의 기술이 4개 모두 중복됩니다. 특히 이전 경험에서 Java/Spring Boot 마이크로서비스를 주도했고(AWS, Docker도 사용) API",
      "locationInsight": "직무는 New York, NY로 명시되어 있지만 isRemote=true입니다. 당신의 현재 위치는 San Francisco, CA이므로 원격 근무 전체라면 지리적 제약이 거의 없습니다.",
      "salaryInsight": "경력 6년 기준으로 제시된 범위(105,000-140,000달러)에 부합할 가능성이 있습니다. 다만 구체 연봉은 명시되지 않았으므로 범위 내 협상 여지가 있습니다.",
      "finalRecommendation": "장점은 Java/Spring Boot 기반 백엔드 마이크로서비스 경험이 직접적으로 매칭되고, Docker, AWS까지 이미 다뤄 성과를 빠르게 낼 가능성이 큼니다. 단점은 Go/gRPC 등"
    },
    {
      "jobId": 2,
      "jobTitle": "Backend Developer - Golang",
      "skillAlignment": "B&D",
      "skillInsight": "요구기술(Go, gRPC, PostgreSQL, Kubernetes)과 당신의 보유기술(Java, Spring Boot, Hibernate, AWS) 간 중복이 거의 없습니다. 마이크로서비스 전반 경험은 있으나, 기술 스택",
      "locationInsight": "직무는 location이 Remote로 표기되어 있으나 isRemote=false입니다. 데이터만 기준으로는 완전 원격이 아닐 수 있어, 당신의 현재 San Francisco 기반으로는 근무 형태(소",
      "salaryInsight": "경력 6년을 가정하면 범위(95,000-125,000달러) 내 협상 가능성이 있습니다. 다만 직무별 구매 연봉은 명시되지 않았으므로 범위 내 협상 여지가 있습니다.",
      "finalRecommendation": "장점은 마이크로서비스를 다룬 경험이 있어 백엔드 역할 적응은 가능할 수 있습니다. 단점은 당신의 핵심 스택이 Java/Spring Boot/AWS 중심인데, 요구기술(Go, gRPC, ..."
    }
  ]
}
```

CareerAdvisorController.java

```
16 public class CareerAdvisorController {
33 }
34
35 /**
36  * 후보자가 선택한 여러 개의 직업에 대해 비교 분석 결과를 반환하는 엔드포인트
37  * 예) http://localhost:8083/api/career-advisor/compare-jobs?candidateId=1&jobIds=1,2,3
38  */
39 @GetMapping("/compare-jobs")
40 public JobsComparisonResult compareJobs(@RequestParam Integer candidateId, @RequestParam List<Integer> jobIds) {
41     log.info("Comparing jobs for candidateId: {}, jobIds: {}", candidateId, jobIds);
42     return this.careerAdvisorService.compareJobs(candidateId, jobIds);
43 }
44 }
```

```

CareerAdvisorService.java × application.properties CareerAdvisorController.java <> index.html <
15 public class CareerAdvisorService {
41
42     public JobsComparisonResult compareJobs(Integer candidateId, List<Integer> jobIds) { 1 usage
43         //1. 후보자 상세 정보를 가져온다.
44         var candidate = this.candidateClient.getCandidateDetails(candidateId);
45         //2. 여러 일자리의 상세 정보를 가져온다. - 결과 List<JobDetails>
46         var jobs = this.jobClient.getJobsDetails(jobIds);
47
48         //3. 후보자정보와 후보자가 선택한 여러 일자리를 비교
49         return this.advisorClient.compareJobs(candidate, jobs);
50     }
51

```

```

CandidateClient.java CandidateDetails.java CareerApplication.java CareerAdvisorClient.java ×
public class CareerAdvisorClient { 6 usages
    public JobsComparisonResult compareJobs(CandidateDetails candidate, List<JobDetails> jobs) { 1 usage
        return this.chatClient.prompt() ChatClientRequestSpec
            .advisors(AdvisorParams.ENABLE_NATIVE_STRUCTURED_OUTPUT)
            .system(this.prompts.compareJobs().system())
            .user(spec -> spec.text(this.prompts.compareJobs().user())
                .param(CANDIDATE, this.toJsonString(candidate))
                .param(JOBS, this.toJsonString(jobs)))
            .call() CallResponseSpec
            .entity(JobsComparisonResult.class);
    }
}

```

### .advisors(AdvisorParams.ENABLE\_NATIVE\_STRUCTURED\_OUTPUT) 의미

⇒ Spring AI에서 LLM(OpenAI, Anthropic 등)의 Native Structured Output 기능을 활성화하기 위한 Advisor 파라미터

"LLM에게 JSON Schema 기반의 구조화된 응답을 생성하도록 요청하고, 그 결과를 JobsComparisonResult.class로 안전하게 변환해줘"

### 왜 필요한가?

일반적으로 AI는 아래처럼 자유로운 텍스트를 반환합니다.

1번 채용공고가 가장 적합합니다.  
이유는 연봉이 높고...

이런 응답은

```

<> Java
.entity(JobsComparisonResult.class)

```

로 변환할 수 없습니다.

## Native Structured Output 사용 시

Spring AI가 내부적으로

```
<> JSON

{
  "recommendedJobId": 1,
  "score": 95,
  "reason": "연봉과 기술스택이 적합"
}
```

와 같은 JSON Schema를 모델에게 전달합니다.

모델은 반드시 해당 형식으로 응답해야 합니다.

그러면

```
<> Java

.entity(JobsComparisonResult.class)
```

가 가능해집니다.

<http://localhost:8083/api/career-advisor/generate-resume?candidateId=1&jobId=1>

> 내가 지원하려는 회사에 맞게 AI가 이력서

The screenshot shows the 'TalentFlow AI' Career Advisor interface. The main section is 'Recommended Jobs' with three job cards:

- Senior Software Engineer - Java** (86% Match): FinEdge Technologies • New York, NY. AI Insight: 귀하는 Java/Spring Boot 기반 백엔드 마이크로서비스를 시니어급으로 설계/개발했고 성능 개선(30%)과 AWS 운영 경험에 있어 기술 적합도가 높습니다. 다만 근무지는 샌프란시스코로 제시되어 있어 뉴욕 현지/전환 가능성 확인이 필요하며, 공고가 요구하는 추가 스택(예: 프레임워크/데이터베이스/클라우드 범위)이 무엇인지에 따라 격차가 생길 수 있습니다.
- Backend Services Developer - Java** (83% Match): BrightTech Consulting • Boston, MA. AI Insight: 귀하는 REST API와 CI/CD(Jenkins) 및 마이크로서비스를 구현한 경력이 있어 'Backend Services Developer - Java'에 직접적으로 부합합니다. 다만 AWS/Docker 중심의 경험은 강점이지만, 보스턴 근무 전환(또는 원격 가능 여부)과 공고의 세부 요구사항(특정 프레임워크/DB-테스트/관측 체계)이 확인되어야 100%에 가까워집니다.
- Java Microservices Developer** (74% Match): 제이테크로스 • 서울, 강남. AI Insight: 귀하는 Java 마이크로서비스 백엔드 역량(Spring Boot/Hibernate/AWS)이 명확해 해당 직무의 핵심 기술과 잘 맞습니다. 다만 근무지가 샌프란시스코에서 서울(강남)로 크게 이동해야 하므로 근무 가능 사정/배치/이주 계획이 가장 큰 격차이며, 언어/로컬 협업 요구가 있다면 추가 확인이 필요합니다.

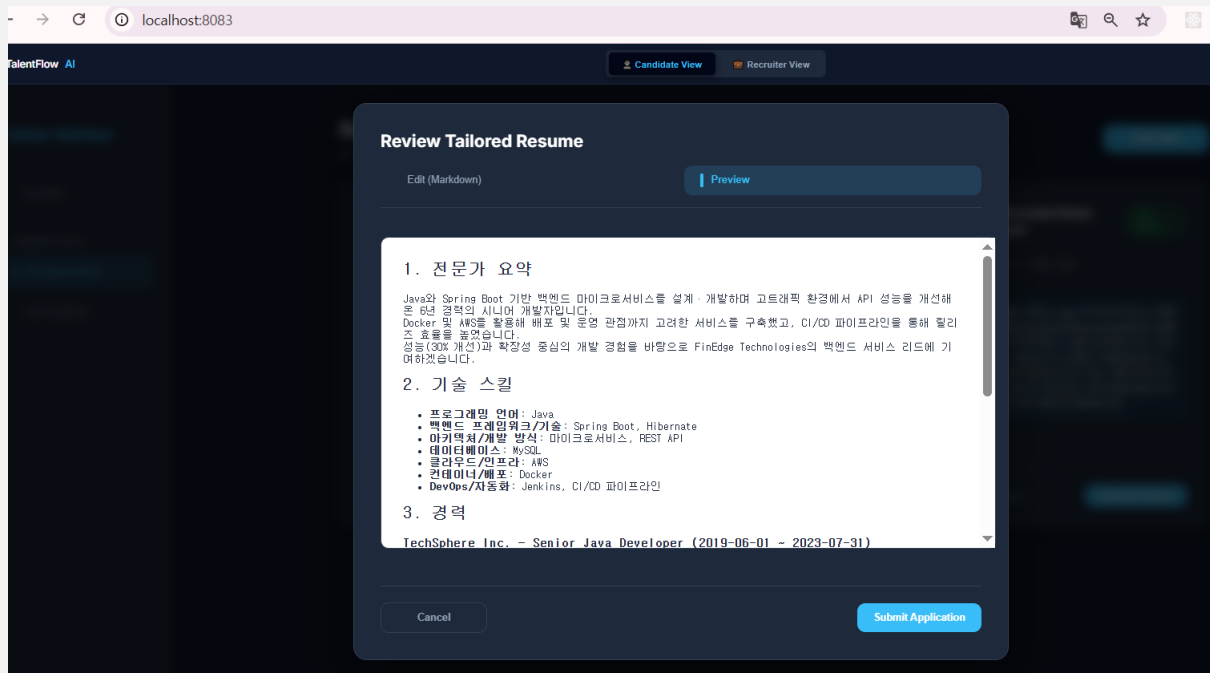
Each card has a 'Compare' checkbox and a 'Generate Resume' button. A red box highlights the 'Generate Resume' button for the first job, with a red arrow pointing to it from the text '클릭' (Click).

career-advisor-cnadiate.html

```
<button class="btn btn-primary btn-compact" onclick="generateResume({{j.jobId}})">
  Generate Resume
</button>
```

```
<script>
async function generateResume(jobId) {
  const cid = document.getElementById('candidate-id').value;
  document.getElementById('modal-title').innerText = "Review Tailored Resume";
  document.getElementById('save-apply-btn').style.display = 'block';
  document.getElementById('resume-modal').style.display = 'flex';
  toggleResumeMode('edit');
  document.getElementById('resume-editor').value = 'AI is crafting your tailored resume...';

  try {
    const res = await fetch(`${ADVISOR_SERVICE_URL}/api/career-advisor/generate-resume?candidateId=${cid}&jobId=${jobId}`);
    const data = await res.json();
    document.getElementById('resume-editor').value = data.resume;
    toggleResumeMode('preview');
    document.getElementById('save-apply-btn').onclick = () => saveAndApply(cid, jobId);
  } catch (e) {
    closeModal();
    alert("Resume generation failed.");
  }
}
}
```



localhost:8083/api/career-advisor/generate-resume?candidateId=1&amp;jobId=1

pretty print 적용 ☒

```
{
  "jobId": 1,
  "candidateId": 1,
  "resume": "## 전문가 요약\nJava와 Spring Boot 기반 백엔드 마이크로서비스를 설계 · 개발하며 고트래픽 혼  
CI/CD 파이프라인을 통해 안정적인 릴리스를 지원했습니다. \n성능(예: API 성능 30% 개선)과 확장성 중심으  
크/기술*: Spring Boot, Hibernate \n- **인프라/클라우드*: AWS \n- **컨테이너/배포*: Docker \n- **  
TechSphere Inc. - Senior Java Developer (2019-06-01 ~ 2023-07-31)\n- 고트래픽 이커머스 플랫폼을 위한 *  
여 사용자 경험과 처리 효율을 향상시켰습니다.\n- **Docker 및 AWS** 환경에서 서비스 배포/운영을 고려한 구  
31)\n- 확장 가능한 **REST API**를 개발하여 서비스 확장성과 유지보수성을 강화했습니다.\n- **Jenkins 기반  
활용한 데이터 연동을 구현했습니다.\n\n## 추가 정보\n- 위치: San Francisco, CA \n- 이메일: 8253jang@da
```

### CareerAdvisorController.java

```
@GetMapping("/{generate-resume")
public TailoredResume generateResume(@RequestParam Integer candidateId, @RequestParam Integer jobId) {
    log.info("Generating resume for candidateId: {}, jobId: {}", candidateId, jobId);
    return this.careerAdvisorService.generateResume(candidateId, jobId);
}
```

### CareerAdvisorService.java

```
public TailoredResume generateResume(Integer candidateId, Integer jobId) { 1 usage
    //1. 후보자 상세 정보를 가져온다.
    var candidate = this.candidateClient.getCandidateDetails(candidateId);

    //2. jobId에 해당하는 일자리 상세 정보 가져온다.
    var job = this.jobClient.getJobDetails(jobId);

    //3. 후보자 정보와 일자리 정보를 기반으로 맞춤형 이력서를 생성한다.
    return this.advisorClient.generateResume(candidate, job);
}
```

### CareerAdvisorClient.java

```

public TailoredResume generateResume(CandidateDetails candidate, JobDetails job) { 1 usage
    var resume = this.chatClient.prompt() ChatClientRequestSpec
        .system(this.prompts.generateResume().system())
        .user(spec -> spec.text(this.prompts.generateResume().user())
            .param(CANDIDATE, this.toJsonString(candidate))
            .param(JOB, this.toJsonString(job)))
        .call() CallResponseSpec
        .content();

    return new TailoredResume(job.id(), candidate.id(), resume);
}

```

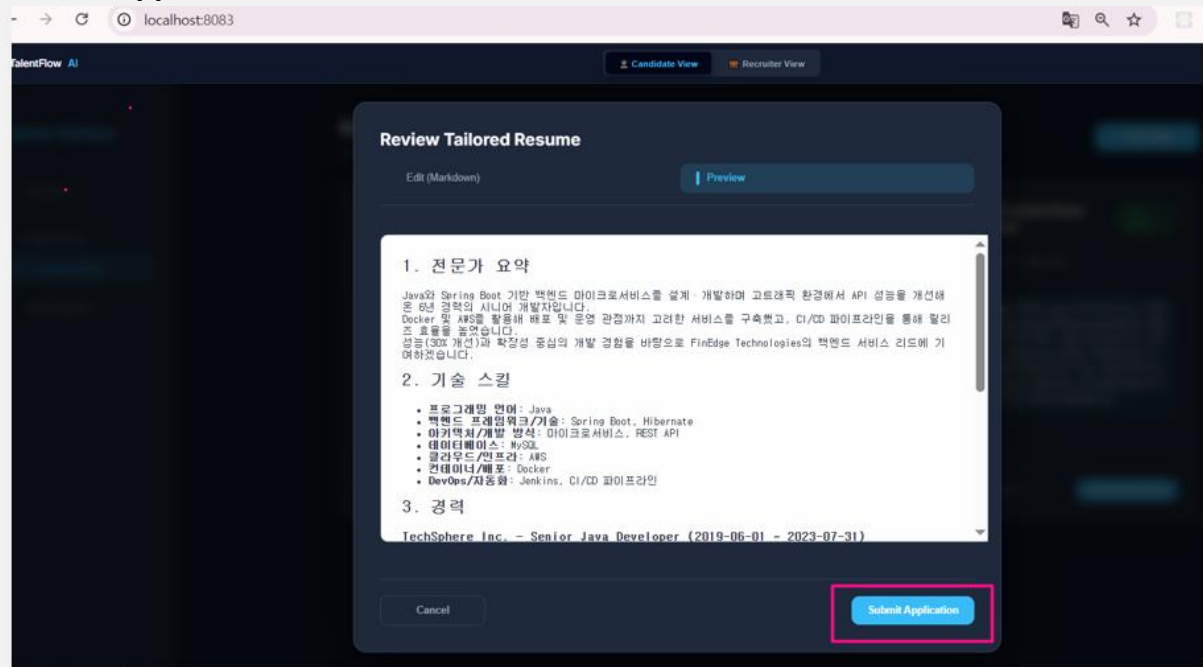
### TailoredResume.java

```

public record TailoredResume(Integer jobId, 6 usages
    Integer candidateId, no
    String resume) { no usage
}

```

### Submit Application(이력서 전송) 기능



### : career-advisor-cadidate.html 문서

```

<div class="card-actions">
    <button class="btn btn-outline" onclick="closeModal()">Cancel</button>
    <button class="btn btn-primary" id="save-apply-btn">Submit Application</button>
</div>

```

```
document.getElementById('save-apply-btn').onclick = () => saveAndApply(cid, jobId);
```

```
async function saveAndApply(cid, jobId) { Show usages
  const content = document.getElementById('resume-editor').value;

  try {
    await fetch(`${HIRING_SERVICE_URL}/api/job-applications`, {
      method: 'POST',
      headers: {'Content-Type': 'application/json'},
      body: JSON.stringify({
        jobId: jobId,
        candidateId: cid,
        resume: content
      })
    });

    showToast("Application Submitted!");
    closeModal();
    showSection('profile');
  } catch (e) {
```

## Application History

TalentFlow AI

Candidate View Recruiter View

Career Advisor

My Profile

ADVISOR TOOLS

Job Opportunities

Job Comparison

장희정

San Francisco, CA  
+1-555-987-6543  
8253jang@daum.net

6 Years Professional Experience

Core Skills

Java Spring Boot Hibernate AWS

Experience History

Senior Java Developer @ TechSphere Inc.  
2019-06-01 - 2023-07-31  
Led the design and development of backend microservices for a high-traffic e-commerce platform using Java and Spring Boot. Improved API performance by 30%.

Backend Developer @ CloudEdge Solutions  
2019-05-01 - 2019-05-31  
Developed scalable REST APIs and implemented CI/CD pipelines.

Application History

| Job ID | Job Title                         | Employer              | Applied Date | Action      |
|--------|-----------------------------------|-----------------------|--------------|-------------|
| 1      | Senior Software Engineer - Java   | FinEdge Technologies  | 2026-06-08   | View Resume |
| 9      | Java Microservices Developer      | 제이드크로스                | 2026-06-09   | View Resume |
| 7      | Backend Services Developer - Java | BrightTech Consulting | 2026-06-09   | View Resume |



## career-advisor-candidate.html

```

async function loadHistory() { Show usages
  const cid = document.getElementById('candidate-id').value;
  const res = await fetch(`${HIRING_SERVICE_URL}/api/job-applications?candidateId=${cid}`);
  const apps = await res.json();

  document.getElementById('history-body').innerHTML = apps.map(a => `
    <tr>
      <td>${a.jobId}</td>
      <td><strong>${a.jobTitle}</strong></td>
      <td>${a.employer}</td>
      <td>${a.appliedDate}</td>
      <td><button class="btn btn-outline" style="..." onclick="viewSentResume('${a.resume.replace(/`/g, '\\`')}')</td>
    </tr>
  `).join('');
}

```

## hiring 프로젝트 JobApplicationController.java

```

@GetMapping(params = "candidateId")
public List<CandidateApplication> getApplicationsByCandidateId(Integer candidateId){
    log.info("Fetching job applications by candidateId: {}", candidateId);
    return this.jobApplicationService.getApplicationsByCandidateId(candidateId);
}

```

## hiring 프로젝트 JobApplicationService.java

```

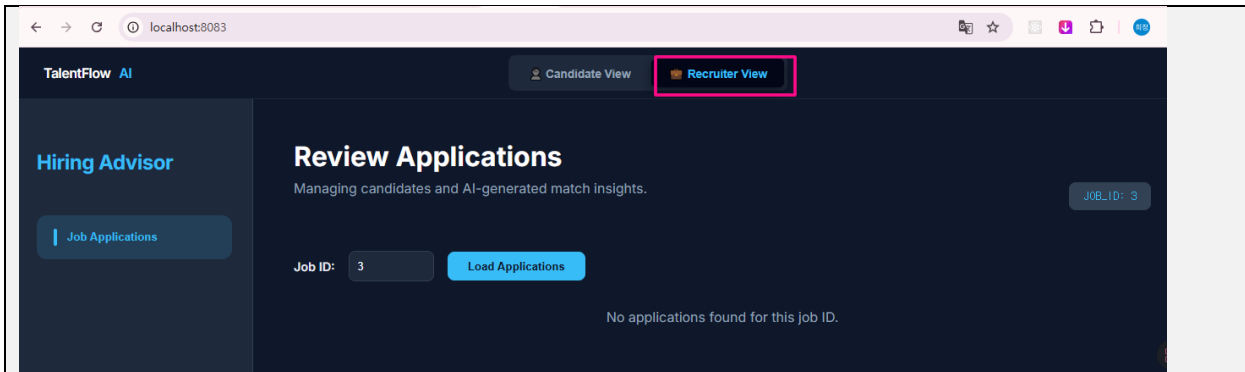
public List<CandidateApplication> getApplicationsByCandidateId(Integer candidateId) { 1 usage
    var jobApplications = this.repository.findByCandidateId(candidateId);
    if (jobApplications.isEmpty()) {
        return Collections.emptyList();
    }
    var jobIds = jobApplications.stream() Stream<JobApplication>
        .map(JobApplication::getJobId) Stream<Integer>
        .toList();

    var jobsMap = this.jobClient.getJobsDetails(jobIds) List<JobDetails>
        .stream() Stream<JobDetails>
        .collect(Collectors.toMap(
            JobDetails::id,
            Function.identity()
        ));

    return jobApplications.stream() Stream<JobApplication>
        .map(jobApplication -> EntityDtoMapper.toCandidateJobApplication(jobApplication, jobsMap.get(jobApplication.getJobId())))
        .toList();
}

```

## Recruiter View



### career-advisor-recruiter.html

```
<div class="controls">
  <span style="...">Job ID:</span>
  <input type="number" id="input-job-id" value="3">
  <button class="btn" onclick="fetchApplications()">Load Applications</button>
</div>
```

```
async function fetchApplications() {
  const jobId = document.getElementById('input-job-id').value;
  document.getElementById('current-job-id').innerText = jobId;
  const container = document.getElementById('applications-container');

  container.innerHTML = '<p style="text-align: center; color: var(--text-muted);">Retrieving enriched applications...</p>';

  try {
    const res = await fetch(`${HIRING_SERVICE_URL}/api/job-applications?jobId=${jobId}`);
    const apps = await res.json();
    renderApplications(apps);
  } catch (e) {
    container.innerHTML = '<p style="color: var(--danger);">Failed to load applications.</p>';
  }
}
```

### REST API으로 hiring 프로젝트 요청

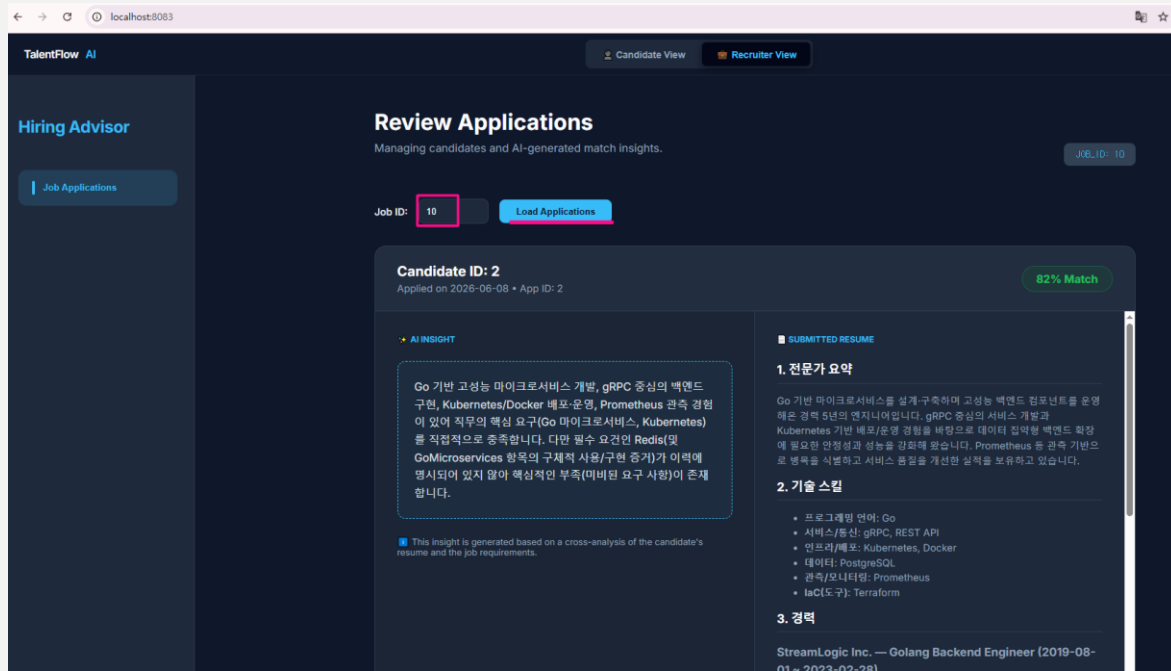
#### hiring프로젝트 JobApplicationController.java

```
@GetMapping(params = "jobId")
public List<JobApplicationDetails> getApplicationsByJobId(Integer jobId){
    log.info("Fetching job applications by jobId: {}", jobId);
    return this.jobApplicationService.getApplicationsByJobId(jobId);
}
```

#### hiring프로젝트 JobApplicationService.java

```
public List<JobApplicationDetails> getApplicationsByJobId(Integer jobId) { 1 usage
    return this.repository.findByJobId(jobId) List<JobApplication>
        .stream() Stream<JobApplication>
        .map(EntityDtoMapper::toJobApplicationDetails) Stream<JobApplicationDetails>
        .toList();
}
```

## 실행결과



## rest-client.http 파일

IntelliJ IDEA Ultimate 버전이라면 **.http** 또는 **.rest** 확장자의 파일을 만들면 자동으로 HTTP Client 기능이 활성화

```
1  ### Find Jobs
2  GET http://localhost:8083/api/career-advisor/find-jobs?candidateId=1
3
4  ### Compare Jobs
5  GET http://localhost:8083/api/career-advisor/compare-jobs?candidateId=1&jobIds=1,2
6
7  ### Generate Resume
8  GET http://localhost:8083/api/career-advisor/generate-resume?candidateId=1&jobId=1
```

## # hiring Project – REST API

Port : 8084

DB : h2 / data.sql제공

### JOB\_APPLICATION

```
CREATE TABLE job_application (
    id SERIAL PRIMARY KEY,
    job_id INT NOT NULL,
    candidate_id INT NOT NULL,
    applied_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    resume TEXT NOT NULL,
    match_score INT,
    match_reasoning TEXT,
    CONSTRAINT uq_job_candidate UNIQUE (job_id, candidate_id)
);
```

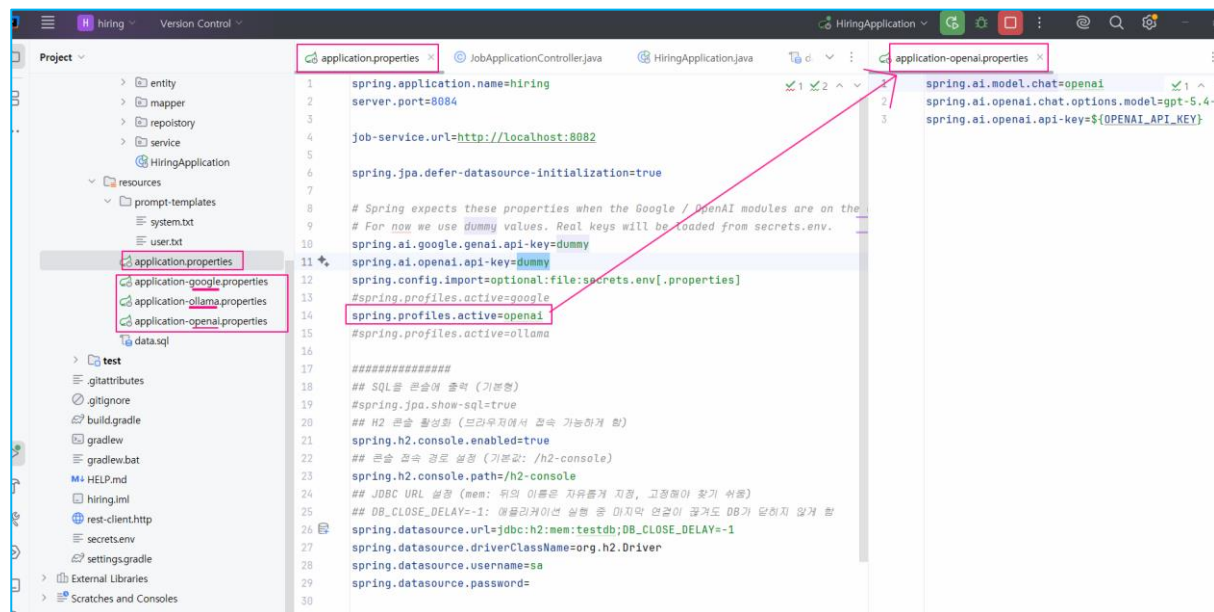
SELECT \* FROM JOB\_APPLICATION;

| ID | JOB_ID | CANDIDATE_ID | APPLIED_DATE | RESUME |
|----|--------|--------------|--------------|--------|
|    |        |              |              |        |

### build.gradle

```
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation 'org.springframework.ai:spring-ai-starter-model-google-genai'
    implementation 'org.springframework.ai:spring-ai-starter-model-ollama'
    implementation 'org.springframework.ai:spring-ai-starter-model-openai'
    implementation 'tools.jackson.core:jackson-databind:3.1.3'
    runtimeOnly 'com.h2database:h2'
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}
```

## application.properties



## JobApplicationRepository.java

```

@Repository 6 usages
public interface JobApplicationRepository extends JpaRepository<JobApplication, Integer> {
    List<JobApplication> findByJobId(Integer jobId); 1 usage

    List<JobApplication> findByCandidateId(Integer candidateId); 1 usage

    Optional<JobApplication> findByJobIdAndCandidateId(Integer jobId, Integer candidateId);

}

```

## 주요기능

### 채용지원서 제출

```

1  ### Submit Job Application (채용 지원서 제출)
2  POST http://localhost:8084/api/job-applications
3  Content-Type: application/json
4
5  {
6    "jobId": 9,
7    "candidateId": 1,
8    "resume": "### 장희정\n**Email:** 8253jang@daum.net | **Phone:** +1-555-987-6543 | **Loc
9  }

```

## JobApplicationController.java

```
@PostMapping
public ResponseEntity<Void> submitApplication(@RequestBody JobApplicationSubmissionRequest request){
    log.info("Submitting job application. jobId: {}, candidateId: {}", request.jobId(), request.candidateId());
    this.jobApplicationService.submitApplication(request);
    return ResponseEntity.accepted().build();
}
```

## JobApplicationService.java

```
@Transactional
public void submitApplication(JobApplicationSubmissionRequest request) {
    var jobApplication = this.repository.findByJobIdAndCandidateId(request.jobId(), request.candidateId())
        .orElseGet(() -> EntityDtoMapper.toJobApplication(request));

    jobApplication.setAppliedDate(LocalDate.now());
    jobApplication.setResume(request.resume());

    // reset previous AI evaluation (will be recomputed async)
    jobApplication.setMatchScore(null);
    jobApplication.setMatchReasoning(null);

    this.repository.save(jobApplication);

    this.eventPublisher.publishEvent(new JobApplicationSubmittedEvent(jobApplication.getId()));
}
```

↓ ApplicationEventPublisher

## JobApplicationEvaluationListener.java

```
@Async
@TransactionalEventListener
public void handle(JobApplicationSubmittedEvent event){
    var jobApplication = this.repository.findById(event.applicationId()).orElseThrow();
    var jobDetails = this.jobClient.getJobDetails(jobApplication.getJobId());

    var evaluationRequest = EntityDtoMapper.toJobApplicationEvaluationRequest(jobApplication, jobDetails);

    var evaluationResponse = this.advisorClient.evaluate(evaluationRequest);
    jobApplication.setMatchScore(evaluationResponse.matchScore());
    jobApplication.setMatchReasoning(evaluationResponse.matchReasoning());

    this.repository.save(jobApplication);
}
```

## DB저장결과

SELECT \* FROM JOB\_APPLICATION;

| ID | JOB_ID | CANDIDATE_ID | APPLIED_DATE        | RESUME                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|----|--------|--------------|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | 1      | 1            | 2026-06-08 00:00:00 | <p><b># 1. 전문가 요약</b><br/>백엔드 마이크로서비스를 Java와 Spring Boot로 설계·개발하며 고트랙픽 환경에서 성능을 개선한 6년 경력의 엔지니어입니다. Docker 기반 배포 및 AWS를 활용해 안정적인 서비스 운영 경험을 보유하고 있으며, API 성능을 30% 향상시킨 성과가 있습니다. CI/CD 파이프라인 구축 경험을 바탕으로 개발-배포 흐름을 효율화해 왔습니다.</p> <p><b># 2. 기술 스킬</b><br/>- **프로그래밍 언어**: Java<br/>- **백엔드/프레임워크**: Spring Boot, Hibernate<br/>- **아키텍처/개발 방식**: 마이크로서비스, REST API<br/>- **데이터베이스**: MySQL<br/>- **클라우드/인프라**: AWS<br/>- **컨테이너/배포**: Docker<br/>- **DevOps/자동화**: Jenkins, CI/CD</p> <p><b># 3. 경력</b><br/><b>### TechSphere Inc. — Senior Java Developer (2019-06-01 ~ 2023-07-31)</b><br/>- Java와 Spring Boot로 **고트랙픽 이커머스 플랫폼 백엔드 마이크로서비스**를 설계·개발하여 서비스 확장성을 강화했습니다.<br/>- API 병목을 분석하고 병렬 처리/응답 구조를 개선해 **API 성능 30% 향상**을 달성했습니다.<br/>- Hibernate 기반 데이터 계층을 적용해 도메인 로직과 영속성 처리를 안정적으로 구현했습니다.<br/>- AWS 및 Docker를 활용해 서비스 운영 및 배포 구성을 정비하여 릴리스 안정성을 높였습니다.<br/>- 기술: Java, Spring Boot, Hibernate, AWS, Docker</p> <p><b>### CloudEdge Solutions — Backend Developer (2016-05-01 ~ 2019-05-31)</b><br/>- 확장 가능한 **REST API**를 개발하고, 서비스 요구사항에 맞춰 엔드포인트 설계를 고도화했습니다.<br/>- Jenkins 기반 **CI/CD 파이프라인**을 구축하여 배포 주기와 운영 효율을 개선했습니다.<br/>- 기술: Java, Microservices, MySQL, Jenkins</p> <p><b># 4. 추가 정보</b><br/>- 없음</p> |

<https://app.notion.com/p/37aa7a6c42ce80fd831bec29d376dab6> 참고

## rest-client.http 파일

IntelliJ IDEA Ultimate 버전이라면 **.http** 또는 **.rest** 확장자의 파일을 만들면 자동으로 HTTP Client 기능이 활성화

```

1  ### Submit Job Application (채용 지원서 제출)
2  POST http://localhost:8084/api/job-applications
3  Content-Type: application/json
4
5  {
6      "jobId": 9,
7      "candidateId": 1,
8      "resume": "### 장희정\n**Email:** 8253jang@daum.net | **Phone:** +1-555-987-6543 | **Location:** San Francisco, CA\n"
9  }
10
11  ### job-applications-by-candidate-id
12  GET http://localhost:8084/api/job-applications?candidateId=1
13
14  ### Fjob-applications-by-job-id
15  GET http://localhost:8084/api/job-applications?jobId=9

```